Table of Contents

..... 1
---------- Settings ---------- 1
---------- 1. Read TLEs -> r, v (GCRF, m and m/s) ---------- 1
---------- 2. Remove duplicate TLEs (same epoch +/- 1 s) ---------- 2
---------- 3. Constants and Earth-pole frame ---------- 2
---------- 4. Select fit and comparison data ---------- 3
---------- 5. Batch weighted least squares (Gauss-Newton) --------- 3
---------- 6. Predict and compare with actual states ---------- 4
---------- 7. Plots ---------- 6
========== Local functions ========== 8


```
close all; clear; clc
% ========================================================================
%  TLE -> state vectors -> batch least-squares orbit fit -> prediction
%  -> comparison with the "actual" (TLE/SGP4-derived) states
%
%  Method: batch weighted least squares (Gauss-Newton differential
%  correction). The unknowns are the satellite state x0 = [r0; v0] at the
%  start of the fit window (plus, optionally, a constant along-track
%  acceleration aT that absorbs drag/low thrust). x0 is propagated with a
%  numerical force model (two-body + J2, J3, J4) and adjusted until the
%  propagated r,v best match all the TLE-derived r,v in the fit window.
%  The fitted state is then propagated past the fit window and compared
%  with the TLE states that were NOT used in the fit.
% ========================================================================
```

---------- Settings ----------

```
tleFile  = "41179_03-01-26_04-01-26.txt";

fitStart = datetime(2026,3,1, 0,0,0,'TimeZone','UTC');   % fit window start
fitEnd   = datetime(2026,3,15, 0,0,0,'TimeZone','UTC');  % fit window end
predEnd  = datetime(2026,3,31,23,0,0,'TimeZone','UTC');  % predict until

estAccel = true;     % also estimate constant along-track acceleration aT
sigR     = 300;      % assumed 1-sigma TLE position error  [m]
sigV     = 0.3;      % assumed 1-sigma TLE velocity error  [m/s]
maxIter  = 15;       % Gauss-Newton iterations
dtDense  = 60;       % step for the dense predicted trajectory [s]
```

---------- 1. Read TLEs -> r, v (GCRF, m and m/s) ----------

```
lines = readlines(tleFile);
i1 = find(startsWith(lines,"1 "));
```

```
N = numel(i1);
temp = [tempname '.tle'];
t = NaT(N,1); t.TimeZone = "UTC";
r = zeros(3,N); v = zeros(3,N);

for k = 1:N
    writelines(lines(i1(k):i1(k)+1),temp);
    tle = tleread(temp);
    t(k) = tle.Epoch;
    [r(:,k), v(:,k)] = propagateOrbit(t(k),tle);
end
delete(temp);

[t, idx] = sort(t);
r = r(:,idx); v = v(:,idx);
```

---------- 2. Remove duplicate TLEs (same epoch +/- 1 s) ----------

```
[~, iu] = uniquetol(seconds(t - t(1)), 1, 'DataScale', 1);
iu = sort(iu);
fprintf('%d TLEs read, %d unique epochs kept\n', N, numel(iu));
t = t(iu); r = r(:,iu); v = v(:,iu);
```

91 TLEs read, 64 unique epochs kept

---------- 3. Constants and Earth-pole frame ----------

```
C.mu = 3.986004418e14;      % m^3/s^2
C.Re = 6378137.0;           % m
C.J2 = 1.08262668e-3;
C.J3 = -2.53265649e-6;
C.J4 = -1.61962159e-6;

% propagateOrbit returns GCRF (J2000-like) vectors, but the Earth's
% oblateness is symmetric about the CURRENT spin axis, which in 2026 is
% tilted ~0.35 deg from GCRF z (precession). Using GCRF z as the J2 axis
% would cause km/day errors, so all fitting is done in a frame whose
% z-axis is the Earth's true pole.
dcm = dcmeci2ecef('IAU-2000/2006', datevec(t(1)));
pole = dcm(3,:).';                          % ITRF z-axis expressed in GCRF
e1 = cross([0;1;0], pole); e1 = e1/norm(e1);
e2 = cross(pole, e1);
R  = [e1.'; e2.'; pole.'];                  % GCRF -> pole frame
rP = R*r;  vP = R*v;
```

---------- 4. Select fit and comparison data ----------

```
iFit  = find(t >= fitStart & t <= fitEnd);
iComp = find(t >= fitStart & t <= predEnd);
if numel(iFit) < 3
    error('Need at least 3 TLE epochs inside the fit window.');
end
tRef = t(iFit(1));                          % epoch of estimated state
tFit = seconds(t(iFit) - tRef);
m    = numel(iFit);
```

---------- 5. Batch weighted least squares (Gauss-Newton) ---------

```
nP = 6 + estAccel;
p  = [rP(:,iFit(1)); vP(:,iFit(1)); zeros(estAccel,1)];  % initial guess
y  = reshape([rP(:,iFit); vP(:,iFit)], [], 1);           % observations
sw = repmat([ones(3,1)/sigR; ones(3,1)/sigV], m, 1);     % sqrt(weights)
step = [10 10 10 0.01 0.01 0.01 1e-8].';                 % finite-diff steps
step = step(1:nP);

fprintf('\nIter   |dr0| [m]    |dv0| [m/s]   wRMS\n');
for it = 1:maxIter
    h   = stateModel(p, tFit, C);
    res = y - h;

    % Jacobian d(h)/d(p) by central differences
    H = zeros(6*m, nP);
    for j = 1:nP
        e = zeros(nP,1); e(j) = step(j);
        H(:,j) = (stateModel(p+e,tFit,C) - stateModel(p-e,tFit,C)) /
(2*step(j));
    end

    dx = (sw.*H) \ (sw.*res);     % weighted normal-equation solution
    p  = p + dx;
    fprintf('%3d  %11.3f  %12.6f  %8.4f\n', it, norm(dx(1:3)), ...
            norm(dx(4:6)), sqrt(mean((sw.*res).^2)));
    % Stop when the weighted RMS stops improving (steps of ~1 m are at the
    % integrator / finite-difference noise floor and will never reach 0)
    wrms = sqrt(mean((sw.*res).^2));
    if it > 1 && abs(wrmsPrev - wrms) < 1e-5*wrms, break; end
    wrmsPrev = wrms;
end

% Final residuals and formal covariance (scaled by a-posteriori variance)
res  = y - stateModel(p, tFit, C);
s2   = sum((sw.*res).^2) / (6*m - nP);
% Columns of H have very different units (m, m/s, m/s^2), so scale them
```

```
% before inverting to avoid an ill-conditioned normal matrix
A    = sw.*H;
D    = diag(1 ./ vecnorm(A));
As   = A*D;
Pcov = s2 * D * ((As.'*As) \ eye(nP)) * D;
sig  = sqrt(diag(Pcov));

fprintf('\nEstimated state at %s UTC (GCRF):\n', string(tRef));
fprintf('  r0 = [%14.3f %14.3f %14.3f] m\n', R.'*p(1:3));
fprintf('  v0 = [%14.6f %14.6f %14.6f] m/s\n', R.'*p(4:6));
fprintf('  1-sigma r0: %.1f %.1f %.1f m,  v0: %.4f %.4f %.4f m/s\n',
sig(1:6));
if estAccel
    fprintf('  along-track accel aT = %.3e +/- %.1e m/s^2\n', p(7), sig(7));
end


Iter   |dr0| [m]    |dv0| [m/s]   wRMS
```

---------- 6. Predict and compare with actual states ----------

```
tCmp = seconds(t(iComp) - tRef);
Xhat = propagateState(p, tCmp, C);
rHat = R.'*Xhat(1:3,:);   vHat = R.'*Xhat(4:6,:);      % back to GCRF
rAct = r(:,iComp);        vAct = v(:,iComp);
tC   = t(iComp);
isFit = tC <= fitEnd;

dr = rHat - rAct;                    % position error  [m]
dv = vHat - vAct;                    % velocity error  [m/s]
drMag = vecnorm(dr);  dvMag = vecnorm(dv);

% Radial / In-track / Cross-track components of the position error
ric = zeros(3, numel(iComp));
for k = 1:numel(iComp)
    uR = rAct(:,k)/norm(rAct(:,k));
    uC = cross(rAct(:,k), vAct(:,k)); uC = uC/norm(uC);
    uI = cross(uC, uR);
    ric(:,k) = [uR uI uC].' * dr(:,k);
end

fprintf('\nFit arc (%d epochs): RMS |dr| = %.3f km, RMS |dv| = %.3f m/s\n',
...
        nnz(isFit), sqrt(mean(drMag(isFit).^2))/1e3,
sqrt(mean(dvMag(isFit).^2)));
fprintf('\nPrediction vs actual (epochs not used in the fit):\n');
fprintf('  %-20s %8s %9s %9s %9s %9s %9s\n', 'Epoch (UTC)', 'dt [d]', ...
        '|dr| km', 'R km', 'I km', 'C km', '|dv| m/s');
for k = find(~isFit).'          % row vector, so the loop runs once per epoch
    fprintf('  %-20s %8.3f %9.3f %9.3f %9.3f %9.3f %9.3f\n', ...
        string(tC(k),'yyyy-MM-dd HH:mm:ss'), days(tC(k)-fitEnd), ...
```

```
        drMag(k)/1e3, ric(:,k)/1e3, dvMag(k));
end

% Dense predicted trajectory (every dtDense seconds)
tDense = (0:dtDense:seconds(predEnd - tRef)).';
Xd = propagateState(p, tDense, C);
rDense = R.'*Xd(1:3,:);
tDenseUTC = tRef + seconds(tDense);
```

Fit arc (28 epochs): RMS |dr| = 0.747 km, RMS |dv| = 0.365 m/s

Prediction vs actual (epochs not used in the fit):

Epoch (UTC)	dt [d]	|dr| km	R km	I km	C km	|dv| m/s
2026-03-15 10:07:26	0.422	1.359	-0.200	0.586	1.210	0.888
2026-03-15 23:16:28	0.970	1.654	-0.213	1.019	1.286	1.349
2026-03-16 17:21:24	1.723	2.198	-0.229	1.659	1.424	2.030
2026-03-17 09:47:41	2.408	2.957	-0.245	2.516	1.535	2.945
2026-03-17 16:22:12	2.682	3.197	-0.244	2.770	1.578	3.221
2026-03-18 17:01:38	3.709	4.518	-0.266	4.157	1.749	4.706
2026-03-18 23:36:09	3.983	4.949	-0.275	4.608	1.783	5.189
2026-03-19 16:02:26	4.668	6.181	-0.290	5.873	1.904	6.543
2026-03-20 03:32:50	5.148	7.265	-0.306	6.986	1.972	7.736
2026-03-20 08:28:44	5.353	7.668	-0.311	7.392	2.015	8.172
2026-03-20 15:03:14	5.627	8.036	-0.313	7.761	2.058	8.570
2026-03-20 21:37:45	5.901	8.689	-0.322	8.427	2.092	9.285
2026-03-21 05:50:54	6.244	9.378	-0.328	9.124	2.144	10.025
2026-03-21 22:17:11	6.929	10.702	-0.340	10.456	2.256	11.457
2026-03-22 04:51:42	7.203	11.371	-0.346	11.130	2.299	12.175
2026-03-22 21:17:59	7.887	12.916	-0.363	12.686	2.403	13.849
2026-03-23 21:57:25	8.915	15.288	-0.385	15.064	2.579	16.399
2026-03-24 04:31:55	9.189	16.082	-0.394	15.862	2.623	17.254
2026-03-24 22:36:50	9.942	17.991	-0.408	17.774	2.756	19.308
2026-03-25 13:24:29	10.559	19.528	-0.419	19.311	2.872	

```
20.963
  2026-03-26 05:50:46    11.244    21.515    -0.433    21.301     2.998
23.090
  2026-03-26 22:17:03    11.929    23.340    -0.445    23.127     3.115
25.046
  2026-03-27 04:51:34    12.202    24.314    -0.451    24.101     3.178
26.085
  2026-03-27 13:04:42    12.545    25.234    -0.456    25.021     3.241
27.072
  2026-03-28 02:13:43    13.093    26.984    -0.467    26.773     3.331
28.942
  2026-03-28 12:05:29    13.504    28.140    -0.473    27.927     3.421
30.182
  2026-03-28 20:18:38    13.846    29.253    -0.482    29.041     3.485
31.369
  2026-03-28 21:57:16    13.915    29.428    -0.484    29.216     3.494
31.557
  2026-03-29 02:53:09    14.120    30.088    -0.485    29.876     3.530
32.261
  2026-03-30 01:53:56    15.079    32.715    -0.500    32.500     3.710
35.074
  2026-03-30 03:32:33    15.148    33.013    -0.501    32.797     3.737
35.390
  2026-03-30 11:45:42    15.490    33.933    -0.504    33.716     3.800
36.380
  2026-03-30 19:58:50    15.833    34.918    -0.513    34.702     3.845
37.437
  2026-03-31 02:33:21    16.106    35.768    -0.517    35.551     3.908
38.344
  2026-03-31 18:59:37    16.791    37.836    -0.528    37.617     4.025
40.560
  2026-03-31 20:38:15    16.860    38.045    -0.531    37.825     4.052
40.782
```

---------- 7. Plots ----------

3-D trajectory

```
figure('Name','Trajectory');
[xs,ys,zs] = sphere(60);
surf(C.Re*xs/1e3, C.Re*ys/1e3, C.Re*zs/1e3, 'EdgeColor','none', ...
     'FaceColor',[0.3 0.5 0.9], 'FaceAlpha',0.4); hold on
plot3(rDense(1,:)/1e3, rDense(2,:)/1e3, rDense(3,:)/1e3, 'r-',
'LineWidth',0.5);
plot3(rAct(1,isFit)/1e3,  rAct(2,isFit)/1e3,  rAct(3,isFit)/1e3,  'ko',
'MarkerFaceColor','k');
plot3(rAct(1,~isFit)/1e3, rAct(2,~isFit)/1e3, rAct(3,~isFit)/1e3, 'gs',
'MarkerFaceColor','g');
axis equal; grid on; xlabel('X [km]'); ylabel('Y [km]'); zlabel('Z [km]');
legend('Earth','Predicted trajectory','TLE states used in fit', ...
       'TLE states (actual) in prediction','Location','best');
```

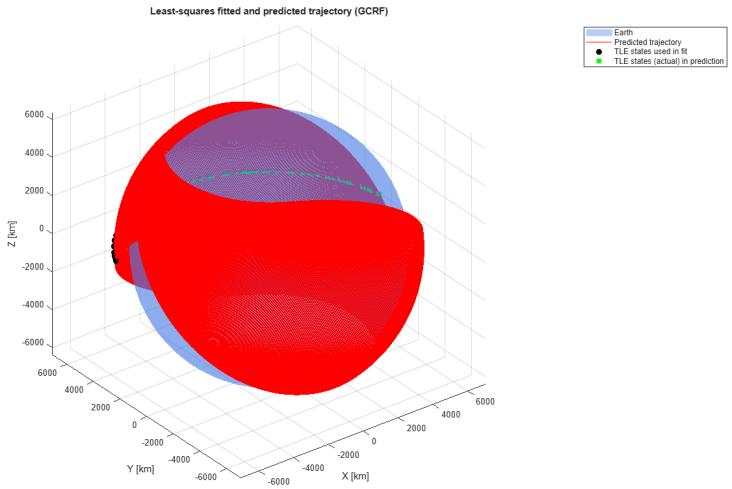
```
title('Least-squares fitted and predicted trajectory (GCRF)');

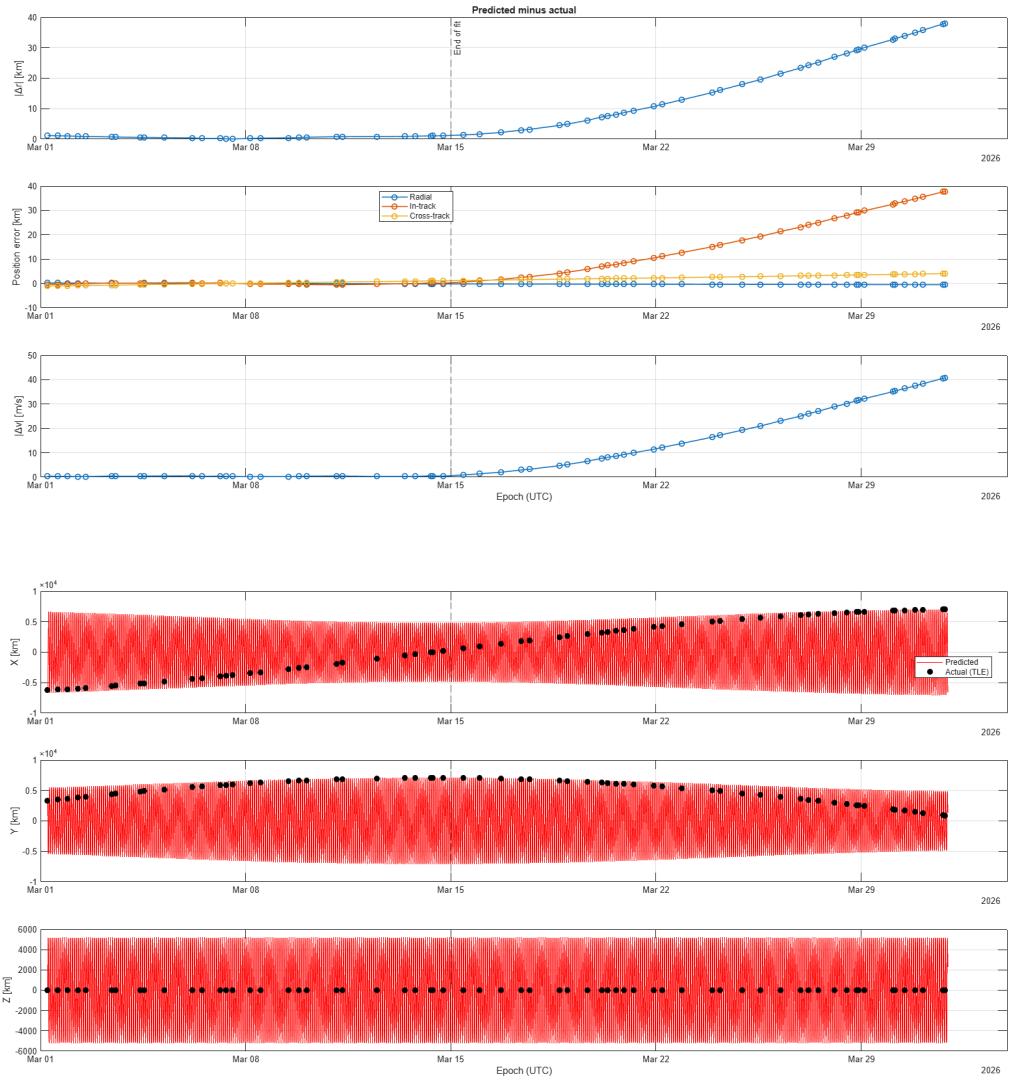
% Position error magnitude
figure('Name','Errors');
subplot(3,1,1)
plot(tC, drMag/1e3, 'o-', 'LineWidth',1.2); grid on
xline(fitEnd, '--', 'End of fit'); ylabel('|\Deltar| [km]');
title('Predicted minus actual');

% RIC components
subplot(3,1,2)
plot(tC, ric/1e3, 'o-', 'LineWidth',1.2); grid on
xline(fitEnd, '--'); ylabel('Position error [km]');
legend('Radial','In-track','Cross-track','Location','best');

% Velocity error magnitude
subplot(3,1,3)
plot(tC, dvMag, 'o-', 'LineWidth',1.2); grid on
xline(fitEnd, '--'); ylabel('|\Deltav| [m/s]'); xlabel('Epoch (UTC)');

% Component-by-component position comparison
figure('Name','Position components');
lbl = {'X','Y','Z'};
for c = 1:3
    subplot(3,1,c)
    plot(tDenseUTC, rDense(c,:)/1e3, 'r-'); hold on
    plot(tC, rAct(c,:)/1e3, 'ko', 'MarkerFaceColor','k');
    xline(fitEnd, '--'); grid on; ylabel([lbl{c} ' [km]']);
    if c == 1, legend('Predicted','Actual (TLE)','Location','best'); end
end
xlabel('Epoch (UTC)');
```





========== Local functions ==========

```
function h = stateModel(p, tOut, C)
% Stacked [r; v] at each output time, as one column (6*m x 1)
    X = propagateState(p, tOut, C);
    h = X(:);
end

function X = propagateState(p, tOut, C)
% Propagate p = [r0; v0; (aT)] from t = 0 to the times tOut [s] (>= 0)
    tOut  = tOut(:);
    tspan = unique([0; tOut]);
    if numel(tspan) == 2                        % ode113 needs >2 points to
        tspan = [tspan(1); mean(tspan); tspan(2)]; % return only these times
```

```
    end
    aT = 0;
    if numel(p) > 6, aT = p(7); end
    opts = odeset('RelTol',1e-11, 'AbsTol',1e-6);
    [~, Y] = ode113(@(tt,x) eom(x, aT, C), tspan, p(1:6), opts);
    [~, loc] = ismember(tOut, tspan);
    X = Y(loc,:).';
end

function dx = eom(x, aT, C)
% Equations of motion: zonal gravity + constant along-track acceleration
    r = x(1:3); v = x(4:6);
    dx = [v; zonalAccel(r, C) + aT * v / norm(v)];
end

function a = zonalAccel(r, C)
% Two-body + J2 + J3 + J4 acceleration, z-axis along Earth's pole
    x = r(1); y = r(2); z = r(3);
    rn = norm(r); z2 = (z/rn)^2;
    a0 = -C.mu * r / rn^3;

    k2 = -1.5 * C.J2 * C.mu * C.Re^2 / rn^5;
    a2 = k2 * [x*(1-5*z2); y*(1-5*z2); z*(3-5*z2)];

    k3 = -2.5 * C.J3 * C.mu * C.Re^3 / rn^7;
    f3 = 3*z - 7*z^3/rn^2;
    a3 = k3 * [x*f3; y*f3; 6*z^2 - 7*z^4/rn^2 - 0.6*rn^2];

    k4 = 15/8 * C.J4 * C.mu * C.Re^4 / rn^7;
    f4 = 1 - 14*z2 + 21*z2^2;
    a4 = k4 * [x*f4; y*f4; z*(5 - 70/3*z2 + 21*z2^2)];

    a = a0 + a2 + a3 + a4;
end
```

```
  1     1168.342       0.367141    71.9661
  2        5.201       0.008066     1.2064
  3        0.601       0.000536     1.1309
  4        1.256       0.001371     1.1309

Estimated state at 01-Mar-2026 05:51:02 UTC (GCRF):
  r0 = [  -6245102.329    3325570.900      14801.843] m
  v0 = [  -2386.596670   -4520.825720    5500.265840] m/s
  1-sigma r0: 70.9 86.1 99.9 m,  v0: 0.1127 0.0834 0.0614 m/s
  along-track accel aT = -3.815e-08 +/- 2.5e-10 m/s^2
```

Published with MATLAB® R2026a